

# 分治

分治算法是信息学竞赛中一个常考的算法，他主要包括二分查找，二分答案，三分查找，CDQ分治，整体二分等等，很多高级的算法和数据结构都含有分治的思想，比如线段树，排序，分块等等。

分治算法的思想是把一个大的问题转换成两个或多个小问题，再把这些小问题继续分割下去，直到可以直接得到答案为止。

洛谷 5021 赛道修建 (NOIP2018)

洛谷 2678 跳石头 (NOIP2015)

洛谷 2680 运输计划 (NOIP2015)

洛谷 1084 疫情控制 (NOIP2012)

洛谷 1083 借教室 (NOIP2012)

洛谷 1314 聪明的质检员 (NOIP2011)

# 二分查找

二分查找可以手写，但是一定要注意边界问题，不然很容易死循环，二分答案有STL，包含在库函数`#include<algorithm>`中。其中三个函数在noip范围内经常被用到：

```
lower_bound();
```

```
upper_bound();
```

```
unique();
```

# algorithm

`lower_bound(a+1, a+n+1, x);`

二分查找有序表中第一个**大于等于**x的数字，如果找到，返回该数字的**地址**，如果没有找到，则返回数组最后一个数的下一个地址。仅适用于**非降序**的有序表，如果是非升序的有序表，则需要重载：

`lower_bound(a+1, a+n+1, x, greater<int>());`

返回有序表中第一个**小于等于**x的数的地址，仅适用于非升序的有序表

# algorithm

一个长度为 $n$ 的有序不下降序列，给定 $m$ 次查找，如果查找的数在序列中，输出在序列中第一次出现的位置，如果不在序列中，输出他在哪两个位置之间(保证查找的数肯定在序列两个端点之间)

```
int t=lower_bound(a+1, a+n+1, x)-a;  
//查找第一个大于等于x的数的位置，返回值是地址  
if(a[t]==x) //如果这个数==x  
printf("%d\n", t);  
else //否则就在两个数之间  
printf("%d %d\n", t-1, t);
```

# algorithm

`upper_bound(a+1, a+n+1, x);`

二分查找有序表中第一个大于x的数的地址，如果找到，返回该数的地址，如果没有找到，则返回末尾数的下一个地址，仅适用于非降序的有序表，如果是非升序的有序表，则需要重载：

`upper_bound(a+1, a+n+1, x, greater<int>());`

返回有序表中第一个小于x的数的位置，仅适用于非升序的有序表

# algorithm

一个长度为 $n$ 的有序不下降序列，给定 $m$ 次查找，如果查找的数在序列中，输出在序列中最后一次出现的位置，如果不在序列中，输出他在哪两个位置之间(保证查找的数肯定在序列两个端点之间)

```
int t=upper_bound(a+1, a+n+1, x)-a;  
//查找第一个大于x的数的位置，返回值是地址  
if(a[t-1]==x) //如果这个数的前一个数==x  
printf("%d\n", t-1);  
else //否则就在两个数之间  
printf("%d %d\n", t-1, t);
```

# algorithm

`unique(a+1, a+n+1):`

STL的去重函数，他的时间复杂度和手动去重(先排序，后去重)一样，都是 $n\log n$ ，但是他的原理和手动去重不一样，他是把重复的元素放到序列的末尾，序列的前 $k$ 个数都是不重复的有效元素，所以输出的时候只需要输出有效长度就可以了。

```
int k=unique(a+1, a+n+1)-a; //得到有效长度
for(int i=1; i<=k; i++) //只输出有效长度
printf("%d ", a[i]);
```

# 二分答案

二分答案和二分查找原理相同，二分查找是枚举位置然后寻找答案，而二分答案是枚举答案然后验证答案是否合理。

二分答案主要用于：

- (1) 答案在一个确定的区间之内，是可以枚举的
- (2) 答案具有单调性，即如果 $x$ 符合条件，那么小于(大于) $x$ 的这些答案一定符合条件；如果 $x$ 不符合条件，那么小于(大于) $x$ 的这些答案一定不符合条件。
- (3) 题目中有很明显的最大值最小或者最小值最大的情况。



# 二分答案

二分答案分为两种，整数二分和实数二分，整数二分的易错点在于 $[x, x+1]$ 二分后的答案永远是 $x$ ，如果直接让 $l=mid$ ，那么就会死循环；实数二分的易错点在于答案一般都是近似值，或者就算是一个浮点数，也不能直接判等，所以要设置一个精度误差，一般比题目要求多2-4位，特别是最后的答案， $l, r, mid$ 的近似值都是一样的，根据做题经验，一般选择 $r$ 作为最后的答案比较好。

# 整数二分

```
//最大值最小
int l=Min;
int r=Max;
while(l<=r)
{
    int mid=(l+r)/2;
    if(check(mid))//如果当前值符合条件
    {
        ans=mid;//先假设就是答案，再查询是否有更小的
        r=mid-1;//mid答案已经记录了，所以用不到了
    }
    else//如果不符合答案，mid肯定不是答案
    {
        l=mid+1;
    }
}
```

# 整数二分

```
// 最小值最大
int l=Min;
int r=Max;
while(l<=r)
{
    int mid=(l+r)/2;
    if(check(mid))//如果当前值符合条件
    {
        ans=mid;//先假设就是答案，再查询是否有更大的
        l=mid+1;
    }
    else//如果不符合答案，mid肯定不是答案
    {
        r=mid-1;//假设的最小值大了，需要缩小
    }
}
```

# 实数二分

```
//最小值最大
double l=Min;
double r=Max;
while(r-l<esp)//esp为精度
{
    double mid=(l+r)/2;
    if(check(mid))
        l=mid;
    else
        r=mid;
}
printf("%.21f",r);
```

# 三分答案

二分答案主要用于答案在一个区间并且具有单调性的问题，那么如果答案在一个区间内，但是不是单增或者单减，而是类似于二次函数一样先增后减或者先减后增呢？

这种情况可以用三分答案来计算极值。三分答案在竞赛中的应用比较少，不过也是一种不错的分治思想。

# 方程的极值

给出一个 $N$ 次函数，保证在范围 $[l, r]$ 内存在一点 $x$ ，使得 $[l, x]$ 上单调增， $[x, r]$ 上单调减。试求出 $x$ 的值。

**输入格式：**

第一行一次包含一个正整数 $N$ 和两个实数 $l$ 、 $r$ ，含义如题目描述所示。

第二行包含 $N+1$ 个实数，从高到低依次表示该 $N$ 次函数各项的系数。

**输出格式：**

输出为一行，包含一个实数，即为 $x$ 的值。四舍五入保留5位小数。

**输入：**

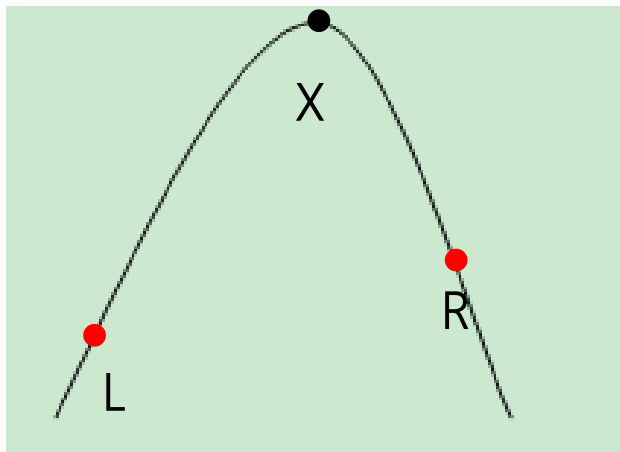
3 -0.9981 0.5

1 -3 -3 1

**输出：**

-0.41421

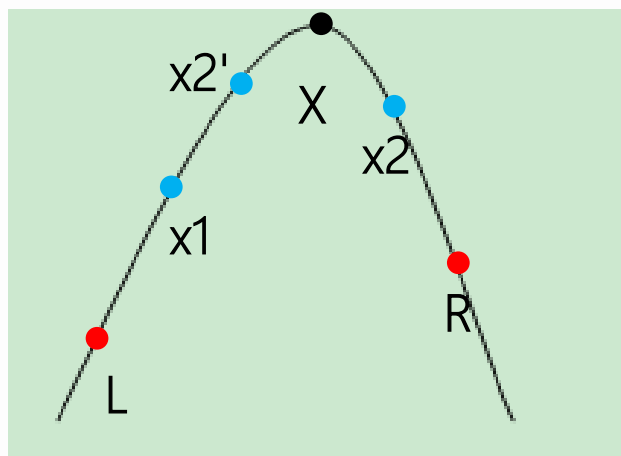
# 方程的极值



首先，这道题的答案（ $x$ 的值）在一个已知的区间内，并且区间肯定是有顺序的，由于是高次方程，所以我们不知道该怎么用公式求解，所以只能一个一个去试答案。

因为要精确到小数点后五位，所以每次增加 $1e-7$ ，效率是比较低的，如果 $l$ 和 $r$ 的值稍微大一点，很容易超时，此时我们就想到了二分答案，找到 $l$ 和 $r$ 的中间点，然后。。。就没有然后了，判断中间点和两端的大小？明显不对。所以二分是不成立的。

# 方程的极值



如果只找一个点，没有与之做比较的，已知该图像在 $[l, r]$ 区间上是严格按照先增后减的规律的，所以我们可以找两个点 $(x_1, x_2)$ ，并且 $x_1 < x_2$ ，然后判断这两个点的函数值的大小，然后缩小极的区间范围。

如果 $f(x_1) < f(x_2)$ ，说明 $x_2$ 更加靠近极值点，因为 $x_1 < x_2$ ，所以 $x_1$ 肯定在极值的左边，但是 $x_2$ 不一定在极值的右边，所以可以缩小范围为 $(x_1, r)$ 。

如果 $f(x_1) > f(x_2)$ ，说明 $x_1$ 更加靠近极值点，因为 $x_1 < x_2$ ，所以 $x_2$ 肯定在极值的右边，但是 $x_1$ 不一定在极值的左边，所以可以缩小范围为 $(l, x_2)$ 。

相等怎么办？



# 方程的极值

接下来是确定该怎么选择这两个点，即满足 $x_1 < x_2$ 的要求，又要让效率更高。即让区间的平均缩小速度最快。很明显就是区间的两个三等分点，这样选择的两个点每次把区间分为以前的 $2/3$ ，多次分割下来，就可以很快找到极值了。这种方法又叫做三分答案。

$$1.5^{35} > 1000000$$

```
int lmid=left+(right-left)/3;//左三等分点
int rmid=right-(right-left)/3;//右三等分点
if(f(lmid)<=f(rmid))//相等情况也是一样的
l=lmid;
else
r=rmid;
```

# 小结

通过前面的讲解，你会发现三分模板特别简单，只要你会二分的模板，三分写起来就很得心应手。但是在实际做题过程中，你会发现，三分最难的地方是你要看出这道题是用三分来求解，也就是说看出我们要求的答案在某个区间内是由极值的，这点是难度最大的。有点难度的题肯定不会直接告诉你或者给你一个方程，都需要我们自己去分析得到结论。

一般情况下，如果答案直接求解求不出来，并且该答案在某一种极限情况下的时候是极大（小）值，再另一种极限情况下的时候也是极大（小）值，那么我们就可以猜测我们需要的答案就是极值点。

# 聪明的质检员

## 题目描述

展开

小T 是一名质量监督员，最近负责检验一批矿产的质量。这批矿产共有  $n$  个矿石，从 1 到  $n$  逐一编号，每个矿石都有自己的重量  $w_i$  以及价值  $v_i$ 。检验矿产的流程是：

- 1、给定  $m$  个区间  $[l_i, r_i]$ ；
- 2、选出一个参数  $W$ ；
- 3、对于一个区间  $[l_i, r_i]$ ，计算矿石在这个区间上的检验值  $y_i$ ：

$$y_i = \sum_{j=l_i}^{r_i} [w_j \geq W] \times \sum_{j=l_i}^{r_i} [w_j \geq W] v_j$$

其中  $j$  为矿石编号。

这批矿产的检验结果  $y$  为各个区间的检验值之和。即：
$$\sum_{i=1}^m y_i$$

若这批矿产的检验结果与所给标准值  $s$  相差太多，就需要再去检验另一批矿产。小T 不想费时间去检验另一批矿产，所以他想通过调整参数  $W$  的值，让检验结果尽可能的靠近标准值  $s$ ，即使得  $|s - y|$  最小。请你帮忙求出这个最小值。

# 聪明的质检员

$$y_i = \sum_{j=l_i}^{r_i} [w_j \geq W] \times \sum_{j=l_i}^{r_i} [w_j \geq W] v_j$$

首先，你要先看懂这个公式代表的意义，该公式是指，对于一个区间 $[l_i, r_i]$ ，检验值 $y_i$ 为：区间内重量 $w_j$ 大于参数 $W$ 的矿石数\*区间内价值 $v_j$ 大于参数 $W$ 的矿石数。

接下来我们考虑 $W$ 的大小与 $y$ 的关系，当 $W$ 增大时，选择的矿石减少，那么 $y$ 就减小；当 $W$ 减小时，选择矿石增加，那么 $y$ 就增大。

最后我们需要的结果是 $|s-y|$ 的绝对值最小。当 $y>s$ 的时候，我们需要减小 $y$ 的值，即增大 $W$ 的值；当 $y<s$ 的时候，我们需要增大 $y$ 的值，即减小 $W$ 的值。所以答案是有单调性的，所以我们考虑二分答案。

# 聪明的质检员

对于每一次假设的 $W$ 的值，我们去计算一次区间的值是 $O(n)$ ，那么 $m$ 次的答案就是 $O(m*n)$ ，明显通不过，我们还需要考虑下如何优化。

实际上我们要求的是 $(a_1*b_1)+(a_2*b_2)+\dots+(a_i*b_i)$ 。我们可以考虑下是否可以把 $a_i$ 和 $b_i$ 在 $O(1)$ 的时间内求出来。

其实方法很简单，当 $W$ 确定之后，我们把重量数组 $w_i$ 都变成01数组，即是否满足条件，这样得到一个01序列，然后我们只需要知道区间有多少个1就可以了，即区间之和，所以我们可以用前缀和，价值 $v_i$ 也是相同的原理。

所以该题就可以用二分答案+前缀和优化来解决。

# 第k小的数

## 题目描述

有两个正整数数列,元素个数分别为  $N$  和  $M$ 。从两个数列中分别任取一个数相乘,这样一共可以得到  $N*M$  个数。

询问这  $N*M$  个数中第  $K$  小数是多少。保证  $K$  合理

## 输入

第一行为三个正整数  $N, M$  和  $K$ 。

第二行为  $N$  个正整数,表示第一个数列。每个数  $\leq \text{int}$

第三行为  $M$  个正整数,表示第二个数列。每个数  $\leq \text{int}$

## 输出

输出文件包含一行,一个正整数表示第  $K$  小数。

# 第k小的数

该题因为数据特别大，所以直接求出所有答案然后做是过不了的。

该题我们很容易知道答案在一个区间，并且对于一个数 $x$ ，如果小于他的数大于 $k$ 个，那么第 $k$ 小的数肯定小于 $x$ ，所以实际上该题是有很明显的单调性的，所以我们可以看考虑二分答案，接下来考虑如何验证。

如果 $k$ 的范围不大，那么我们可以直接验证，先把两个序列排序，然后依次从1向后面枚举，如果小于 $x$ 的数超过了 $k$ 个，那么直接跳出，最多枚举 $k$ 次就可以验证答案了。

但是如果 $k$ 特别大呢？

# 数列

## 题目描述

给定一个长度为  $n$  的非负整数序列  $A$ ，求一个平均数最大的，长度不小于  $L$  的子段。 $L \leq n$

## 输入

第一行用空格分隔的两个整数  $n$  和  $L$ ；

第二行为  $n$  个用空格隔开的非负整数，表示  $A_i$ 。

## 输出

输出一个整数，表示答案的 1000 倍。不用四舍五入，直接输出。



# 数列

该题看上去和最大子串和有点类似，实际上做法完全不一样。首先是平均数最大，并不代表和最大，其次还要满足长度大于1。

我们可以二分平均值，然后把序列看做正反序列，即都减去这个平均值，如果该序列中存在一段之和为非负整数，并且满足长度大于1，那么就说明存在。或者，先满足长度大于1，再判断是否存在非负整数。

区间之和，我们可以看做前缀和相减，最大子串之和，我们可以看做前缀和—最小前缀和，即前 $i$ 个数的前缀和—前 $i-1$ 个数的最小前缀和，那么得到的一定是一个满足长度 $\geq 1$ 并且最大的一个区间和。如果所有的这样的区间都满足条件，那么就说明符合条件。

# 货物收集

## 题目描述

Venn想要收集一些货物。

Venn有一颗 $n$ 个节点的树，一开始Venn在1号节点，其他每个节点都有一定的货物储备，Venn只要经过那些节点，就可以收集到节点的所有货物。每个节点的货物只能收集一次。

显然，Venn并不能轻易的收集所有的货物。每一条连接着两个节点的路径，都有一个邪恶的怪物镇守。Venn的武力值必须不小于怪物的武力值才能安全地从这条路径上通过。

Venn一开始的武力值是0，但是她可以选择健身来提升自己的武力值。每健身一分钟，就会提升一点武力值。Venn并不想收集所有的货物，只要最终收集到的货物总量不低于 $W$ 就可以了。Venn一旦开始收集，就不能再健身了。但是Venn的速度很快，可以认为收集货物和从路径上经过都不需要时间。

由于Venn还急着去颓废，所以她想让你帮她计算收集到指定数量的货物最少需要几分钟。



# 货物收集

该题相对来说比较简单，因为一旦当我们知道他的初始武力值，那么哪些点可以经过，哪些点不可以经过就一目了然了，所以我们二分枚举健身的时间然后去贪心的手机货物最后去验证答案就可以了



# Intelligence test

## Description

霸中智力测试机构的一项工作就是按照一定的规则删除一个序列的数字，得到一个确定的数列。Lyx很渴望成为霸中智力测试机构的主管，但是他在这个工作上做的并不好，俗话说熟能生巧，他打算做很多练习，所以他希望你写一个程序来快速判断他的答案是否正确。

## Input

第一行为一个整数 $m$  ( $1 \leq m \leq 1000000$ ) 第二行包括 $m$ 个用空格分开的整数 $a_i$  ( $1 \leq a_i \leq 1000000$ )，组成了最初的序列，第三行为一个整数 $n$  ( $1 \leq n \leq 1000000$ )，表示 $n$ 个Lyx经过一系列删除得到的序列，每个序列两行，第一行给出长度 $L$  ( $1 \leq L \leq m$ )，然后下一行为 $L$ 个由空格分开的整数 $b_i$  ( $1 \leq b_i \leq 1000000$ )。

## Output

共 $n$ 行，如果Lyx的序列确实是由最初的序列删除一些数得到，就输出TAK，否则输出NIE。

# Intelligence test

简单来说该题就是判断一个字符串是否是另一个字符串的子序列。

很容易想到一种暴力的做法，维护两个指针，一个指向长串，一个指向短串，然后依次匹配，如果可以匹配，那么指针同时右移，如果不可以匹配，那么短串指针不动，长串指针后移。但是这种暴力做法时间复杂度每一次匹配是 $O(m)$ 的，那么多次匹配就是 $m*n$ ，效率不高。

# Intelligence test

实际上对于短串的每一个数字，我们只需要去长串中找匹配的位置就可以了，没有必要一个一个枚举，所以我们完全可以记录每一个数字下一个字符的位置。那么对于短字符串的一个字符，长字符串中这个字符出现的位置我们都知道了，只需要去枚举长字符串的这些该字符出现的位置就可以了？

那么有必要去一个一个枚举吗？并不需要，因为这些位置是有序的，所以我们只需要看看上一次匹配到的位置之后是否和满足条件的位置就可以了，如果有，匹配最小的一个，如果没有，表示不可以匹配。

即第一个数字是a，那么找到长串中数字a出现的第一个位置k，第二个字符b，再去长串中[k,m]之间去查找b出现的第一个位置。二分查找。

# Vector

头文件 `#include<vector>`

向量的定义：

`vector<int> vec;` //定义一个vec型的向量a

`vector<int> vec(5);` //定义一个初始大小为5的向量

`vector<int> vec(5, 1);` //初始大小为5，值都为1的向量

因为vector是可以根据插入的值的数量自动增长的，所以初始定义的时候只需要按照**第一种**方法定义就可以了（一维）。

很多时候我们需要开一个二维数组：

`vector<vector<int>> vec(100);`

# Vector 操作

Vector的下标和数组一样从0开始的

`vec.size()`; //返回向量的实际大小

`vec.begin()`; //返回向量的开始指针的位置

`vec.end()`; //返回向量的结束指针的下一个位置

`vec.push_back(x)`; //在对象末尾插入数据x

`vec.pop_back()`; //在对象末尾删除数据

`vec.clear()`; //清除对象中的所有数据

`vec.at(i)`; //访问容器中第i个数的值(推荐)

`vec[i]`; //访问容器中第i个数的值(不推荐)



# Vector

```
#include<vector>
using namespace std;
vector<int> a;
int main()
{
    int n,m,tmp;
    scanf("%d%d",&n,&m);
    for(int i=1;i<=n;i++)
    {
        scanf("%d",&tmp);
        a.push_back(tmp); //插入操作
    }
    sort(a.begin(),a.end()); //排序, 不能用a+1,a+n+1
    for(int i=1;i<=m;i++)
    a.pop_back(); //删除操作
    for(int i=0;i<a.size();i++)
    printf("%d ",a.at(i)); //访问a中第i个元素
    // printf("%d ",a[i]);
    printf("\n");
}
```

# Vector

在第 $i+1$ 个数前面插入一个数 $x$ :

```
vec.insert(vec.begin()+i, x);
```

删除第 $i+1$ 个数:

```
vec.erase(vec.begin()+i);
```

这才是vector的精髓所在，以上删除，插入操作都是 $\log n$ 的，因为vector下标是从0开始的，所以下标为 $i$ 的数实际上就是第 $i+1$ 个数

以上两个操作再集合两个STL函数可以1行代码实现一个平衡树的单点操作。

# 必做练习题

先去MSOJ刷两道二分查找，三道二分答案的水题

洛谷 1314 聪明的质监员

MSOJ 1435 第k小的数

MSOJ 1433 数列

MSOJ 1665 货物收集

MSOJ 1556 期末考试

洛谷 3500 Intelligence